

Take-Home Full-Stack Assessment

Background Job Runner

You are building a platform that lets people kick off slow background work — bulk scrapes, file conversions, report generation — and watch it run to completion without ever blocking on it. A user submits a job, gets an immediate acknowledgement and a short handle to refer to it by, and moves on. The platform runs the work in the background, shows everything in flight in an operations dashboard, announces the moment a job finishes, and lets the user retrieve results or cancel work they no longer want.

The orchestration engine is the centerpiece — the part that accepts a job, hands back a handle instantly, runs the work in the background under a concurrency limit, tracks each job's state through to completion, and emits an event the moment it finishes. It must be correct under concurrency, durable across restarts, and cleanly separated from the UI behind a documented API. Think Sidekiq, Celery, or Temporal — but small, observable, and yours.

This is a proof of concept, not a production system. We have intentionally left many decisions to you — how you make them is part of what we are evaluating.

Time expectation: ~4-5 days with effective use of AI coding tools.

Recommended Stack

Layer	Recommendation
Frontend	React or any React-based framework (Next.js, Remix, etc.)
Backend	Node.js or Python — pick one and keep the engine in it
Database	Any persistent storage (PostgreSQL, SQLite, Redis, MongoDB, etc.)
Infrastructure	Any host that supports a long-running backend process

What the system does

Submitting jobs

A user submits a job by choosing a category and providing parameters. The platform assigns a handle and returns it **immediately** — before any work runs. The submit path must never block on the job itself; an acknowledgement that takes as long as the job defeats the entire point of the tool.

Handles

Every job gets a short, human-friendly handle made of its category and a number — `scrape-1`, `convert-1`, `report-2`. Numbers are assigned per category, so `scrape-1`, `convert-1`, and `report-1` can all exist at once. A number is recycled once its job is collected or cancelled: after the user retrieves the result of `scrape-1` (or cancels it), the next new scrape job may reuse `scrape-1`. A handle must never collide with an active job — if `scrape-1` is still queued, running, or finished-but-uncollected, a new scrape job gets `scrape-2`. Handle assignment and recycling are part of the engine and are evaluated closely.

The orchestration engine

When a job is submitted it enters a queue. A worker pool pulls from the queue, respecting a configurable concurrency limit, runs the matching worker for the job's category, records the outcome, and emits a completion event. Jobs move through a clear lifecycle — `queued`, `running`, `ready`, `failed`, `cancelled` — and the engine is the single source of truth for that state.

Two things must hold:

- **Non-blocking enqueue.** Submitting a job returns its handle well before the work finishes. The UI stays fully responsive while any number of jobs run.
- **Durability.** Engine state survives a restart. Enqueue jobs, restart the backend process, and the tasks and their statuses are still there, with in-flight work resumed or cleanly re-queued. Sketch your approach in the architecture writeup.

Workers

Workers are where the actual job work happens, and the engine must treat them as pluggable — it hands a worker a job and gets back a result or an error, with no category-specific logic baked into the engine itself. So you can build and demo without real scraping or conversion, implement a **mock worker** that sleeps for a duration given in `params.duration_ms` (default to a random 3–15 seconds) and fails when `params.fail` is true, returning a retryable error. Wire up at least two categories (e.g. `scrape` and `report`) backed by the mock worker. Adding one genuinely real worker is welcome but not required and not where the points are.

Live updates and notifications

The dashboard updates in real time over a server-sent event or WebSocket stream — no refreshing to see a status change. The moment a job finishes, the user gets a clear completion notification surfaced without any action on their part.

Cancellation

A user can cancel a job that is queued or running. A running worker must actually stop when its job is cancelled, not finish silently in the background.

Failure handling

Transient failures (a worker throwing on a flaky operation) are auto-retried with backoff. Permanent failures (a job marked non-retryable, repeated exhaustion of retries) are surfaced and not auto-retried — the operator decides when to retry. Distinguish the two with distinct, honest reasons rather than a generic "failed."

Operations dashboard

Every submitted job appears as an item with its lifecycle status. The dashboard surfaces every item with filters (category, status, date range) and sort. Per-item detail shows the job's parameters, its state history with timestamps, attempt count, the full error message on failure, and the result on success. Operators can act on items: retry a failed job, cancel a queued or running one, and collect a finished result.

The API

The REST API is the product's backbone, not an afterthought — the dashboard is one client of it, and external scripts or agents are another. Expose, at minimum, the endpoints below, authenticated with a per-user API key scoped to that user's data, and document it in the README.

POST	/tasks	enqueue a job; returns the task object immediately
GET	/tasks	list tasks; supports ?status= and ?lane= filters
GET	/tasks/{handle}	fetch one task
GET	/tasks/{handle}/result	fetch the result; marks the task collected
POST	/tasks/{handle}/cancel	cancel a queued or running task
GET	/events	SSE/WebSocket stream of lifecycle events

Task object (field names and shapes are fixed):

```
{
  "handle": "scrape-1",
  "lane": "scrape",
  "params": { },
  "status": "queued",
  "result": null,
  "error": null,
  "created_at": "2026-05-30T18:00:00Z",
  "updated_at": "2026-05-30T18:00:00Z",
  "collected": false
}
```

status is one of **queued**, **running**, **ready**, **failed**, **cancelled**. **result** is populated only when **status** is **ready**; **error** only when **failed**, with the shape `{ "reason": "string", "retryable": true }`. **collected** flips to true once the result is retrieved.

Event shapes:

```
{ "type": "accepted", "handle": "scrape-1", "lane": "scrape", "summary": "..." }
{ "type": "ready", "handle": "scrape-1", "lane": "scrape", "summary": "..." }
{ "type": "failed", "handle": "scrape-1", "lane": "scrape", "reason": "...",
  "retryable": true }
{ "type": "cancelled", "handle": "scrape-1", "lane": "scrape" }
```

Worker contract:

```
interface Job { handle: string; lane: string; params: Record<string, unknown>; }
interface WorkerResult {
  status: "ready" | "failed";
  result?: unknown;
  error?: { reason: string; retryable: boolean };
}
type Worker = (job: Job) => Promise<WorkerResult>;
```

Success criteria

These are concrete checks your submission should pass. They are written as input plus expected observable outcome, and good submissions turn them into automated tests. Your reviewer will walk this list against your deployed app and your test suite. The mock worker reads `duration_ms` and `fail` from a job's params, which makes every scenario below reproducible.

1. **Instant handle.** Submit a `scrape` job with `duration_ms: 10000`. The `POST /tasks` response returns `{ "handle": "scrape-1", "status": "queued" }` in well under one second, far less than the 10s the job will take. The handle is in hand long before the work is done.
2. **Lifecycle to ready.** For that same `scrape-1`: it shows `running` within about a second, then `ready` after about 10s. `GET /tasks/scrape-1/result` returns the result and flips `collected` to true. The `/events` stream carried an `accepted` then a `ready` event for `scrape-1`.
3. **Per-category numbering.** Submit one `scrape` and one `report` job. They get `scrape-1` and `report-1`, not `scrape-1` and `scrape-2`. Submit a second `scrape` while the first still runs: it gets `scrape-2`.
4. **Recycling without collision.** Run `scrape-1` to completion and collect it, then submit a new scrape job: it may reuse `scrape-1`. But while a `scrape-1` is still queued, running, or finished-but-uncollected, a newly submitted scrape job must be `scrape-2`. A handle is never reused while its previous owner is still active.

5. **Failure surfaces, no auto-collect.** Submit a job with `fail: true`. It lands in `failed` with `error.retryable` set and a reason; the `/events` stream carried a `failed` event; the job is not silently collected; the operator can retry it.
6. **Cancellation, running and queued.** Submit a job with `duration_ms: 30000` and cancel it mid-run: it moves to `cancelled`, no later `ready` event arrives for it (the worker actually stopped), and `/events` carried a `cancelled` event. Repeat for a job still sitting in the queue behind the concurrency limit: same result.
7. **Concurrency respected.** With concurrency set to 2, submit 5 jobs at once. At no moment are more than 2 in `running`; the rest stay `queued` and start only as slots free up.
8. **Concurrent completions.** Submit 3 jobs that all finish within about a second of each other. All 3 reach `ready`, all 3 `ready` events arrive on the stream, and no state is lost, duplicated, or left stuck in `running`.
9. **Durability across restart.** Submit several jobs so some are running and some queued, kill the backend process, and restart it. `GET /tasks` still shows every job with its correct status; in-flight work is resumed or cleanly re-queued; nothing is lost. This and the no-collision check are the two we verify most carefully.

Quality bar

The engine must be a standalone module behind the documented API, and the frontend must be a pure consumer of it — no importing engine internals, no reading the engine's database directly. We should be able to delete the frontend and still exercise the engine fully through the API. A well-built engine is reusable across many clients, and we care that the boundary is clean.

The entire interface must be mobile-friendly across every surface — submitting jobs, the dashboard, item detail. The UI must never block while jobs run.

Automated tests are required, not a bonus. At minimum, prove against the API: that enqueue returns a handle before the job finishes; that handle numbering is per-category, recycles after collection, and never collides with an active job; that a job runs to `ready` and its result is retrievable; that a failing job lands in `failed` with a retryable flag; that cancellation works for both queued and running jobs; that concurrent completions are all handled cleanly; and that engine state survives a restart. The no-collision and restart-survival tests are the two we look at hardest.

Real seed data — synthetic jobs in every status so the dashboard's filtering, sorting, and detail views can be demonstrated meaningfully. Clearly distinguish seed data from real processing.

Thoughtfulness over completeness. A polished, considered submission that nails the core flow and treats edge cases with care will beat a feature-complete one that is brittle.

Deliverables

- **GitHub repository** with a clear README covering: architecture, local setup, how to run the engine and the frontend, how the worker pool and mock worker are configured, how to run the tests, and seeding instructions. The project must run in GitHub Codespaces.
- **Deployed URL** with the full flow working end to end — submit a job, watch it run to completion live, retrieve the result, and hit the REST API directly with an API key.
- **Recorded demo (15–30 minutes)** in two parts. First, the working system: submitting jobs across categories, the dashboard updating live, completion notifications, cancelling a running job, a failure-and-retry scenario, restart survival (kill the backend, bring it back, show state intact), and calling the REST API directly. Second, a detailed architectural walkthrough: talk us through the orchestration engine design, the handle assignment and recycling strategy, your concurrency model, the durability and restart strategy, the worker contract and how new workers plug in, the key tradeoffs you weighed, and what you would change with more time. We want to hear your reasoning, not just see the result.

Evaluation

We weigh (the engine first, and evaluated hardest):

- **Orchestration correctness** — instant handle on enqueue, correct state transitions, concurrency respected, completion events fire reliably
- **Handle logic** — per-category numbering, recycling, no collisions with active jobs, all proven by tests
- **Durability and robustness** — state survives restart; clean handling of concurrent completions, cancellation mid-run, and worker failures
- **API and module boundary** — endpoints, shapes, and event types match the contract; engine standalone, frontend a pure consumer
- **Tests** — coverage of the cases in the quality bar, readable and meaningful
- **Operations dashboard clarity** — honest, observable system state; recoverability from common failures
- **Mobile usability** across every surface
- **End-to-end reliability** — working deployment, no silent failures

The best submissions show real polish and judgment in every detail.